ASSIGNMENT 11

Name: Rohit Kailas Deore
Roll No: 11
Batch: T1
Class: TE-IT

[1]:
```
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score,␣
  ↪GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
from sklearn.metrics import accuracy_score

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

1. Load the Dataset

[2]:
```
# Built-in dataset: no Kaggle download is needed
data = load_iris()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target)

print("First five rows:")
display(X.head())
print("Dataset shape:", X.shape)
print("Target classes:", list(data.target_names))
```

```
First five rows:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

```
Dataset shape: (150, 4)
Target classes: [np.str_('setosa'), np.str_('versicolor'), np.str_('virginica')]
```

2. Split the Dataset

[3]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (120, 4)
Testing data: (30, 4)
```

3. Train the Models Before Optimization

[4]:
```
# SVM uses scaling inside a pipeline
svm_model = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", SVC())
])

rf_model = RandomForestClassifier(random_state=42)
ada_model = AdaBoostClassifier(random_state=42)

models = {
    "SVM": svm_model,
    "Random Forest": rf_model,
    "AdaBoost": ada_model
}

before_results = []

for name, model in models.items():
    model.fit(X_train, y_train)
    prediction = model.predict(X_test)
    accuracy = accuracy_score(y_test, prediction)
    before_results.append({"Model": name, "Test Accuracy Before Optimization":␣
  ↪accuracy})
    print(name, "accuracy:", round(accuracy, 4))

before_df = pd.DataFrame(before_results)
display(before_df.round(4))
```

```
SVM accuracy: 0.9667

Random Forest

 accuracy: 0.9

AdaBoost accuracy: 0.9333
```

```
           Model  Test Accuracy Before Optimization
0            SVM                             0.9667
1  Random Forest                             0.9000
2       AdaBoost                             0.9333
```

4. Perform 5-Fold Cross-Validation

[5]:
```
cv_results = []

for name, model in models.items():
    scores = cross_val_score(model, X_train, y_train, cv=5, scoring="accuracy")
    cv_results.append({
        "Model": name,
        "Mean CV Accuracy": scores.mean(),
        "CV Standard Deviation": scores.std()
    })
    print(name, "5-fold CV scores:", scores.round(4))
    print("Mean CV accuracy:", round(scores.mean(), 4))

cv_df = pd.DataFrame(cv_results)
display(cv_df.round(4))
```

```
SVM 5-fold CV scores: [0.9167 1.     0.9583 0.9583 1.    ]
Mean CV accuracy: 0.9667

Random Forest 5-fold CV scores: [0.9167 0.9583 0.9583 0.9583 0.9583]
Mean CV accuracy: 0.95

AdaBoost 5-fold CV scores: [0.9167 0.9583 0.9583 0.9583 0.9583]
Mean CV accuracy: 0.95
```

```
           Model  Mean CV Accuracy  CV Standard Deviation
0            SVM            0.9667                 0.0312
1  Random Forest            0.9500                 0.0167
2       AdaBoost            0.9500                 0.0167
```

5. Optimize SVM Using Grid Search

[6]:
```
# Grid Search tests different SVM settings using 5-fold cross-validation
param_grid = {
    "classifier__C": [0.1, 1, 10],
    "classifier__kernel": ["linear", "rbf"],
    "classifier__gamma": ["scale", "auto"]
}

grid_search = GridSearchCV(
    estimator=Pipeline([
        ("scaler", StandardScaler()),
```

```
        ("classifier", SVC())
    ]),
    param_grid=param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

print("Best SVM parameters:", grid_search.best_params_)
print("Best cross-validation accuracy:", round(grid_search.best_score_, 4))
```

```
Best SVM parameters: {'classifier__C': 0.1, 'classifier__gamma': 'scale',
'classifier__kernel': 'linear'}
Best cross-validation accuracy: 0.975
```

6. Compare SVM Before and After Optimization

[7]:
```
# Evaluate the optimized SVM on the same held-out test set
optimized_svm = grid_search.best_estimator_
optimized_prediction = optimized_svm.predict(X_test)
optimized_accuracy = accuracy_score(y_test, optimized_prediction)

original_svm_accuracy = accuracy_score(
    y_test, models["SVM"].predict(X_test)
)

comparison = pd.DataFrame({
    "Model": ["SVM before optimization", "SVM after Grid Search"],
    "Test Accuracy": [original_svm_accuracy, optimized_accuracy]
})

display(comparison.round(4))
print("Accuracy change:", round(optimized_accuracy - original_svm_accuracy, 4))
```

	Model	Test Accuracy
0	SVM before optimization	0.9667
1	SVM after Grid Search	0.9333

```
Accuracy change: -0.0333
```

7. Final Model Comparison

[8]:
```
# Compare the three original models with the optimized SVM
final_results = pd.DataFrame({
```

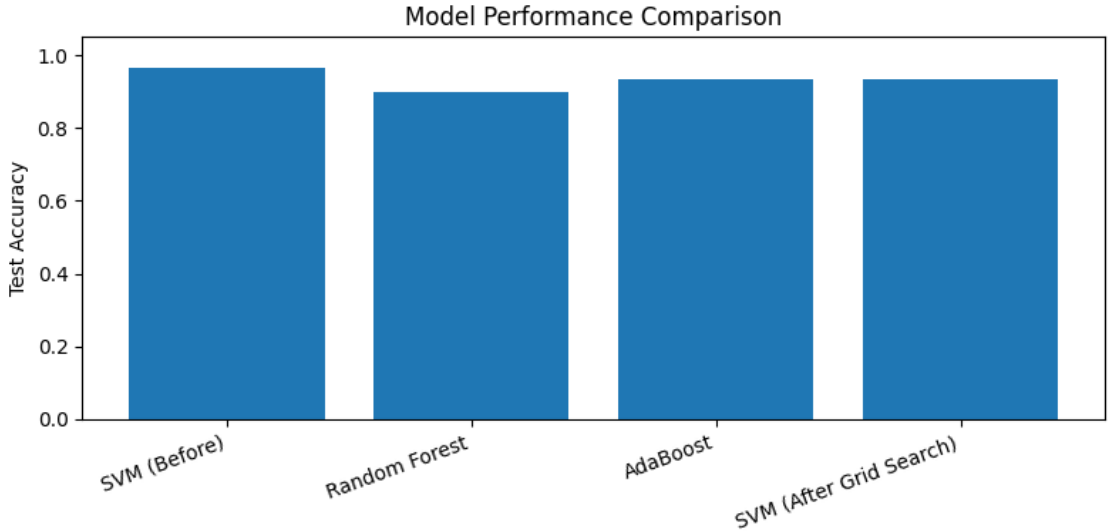
```
    "Model": ["SVM (Before)", "Random Forest", "AdaBoost", "SVM (After Grid␣
  ↪Search)"],
    "Test Accuracy": [
        original_svm_accuracy,
        accuracy_score(y_test, models["Random Forest"].predict(X_test)),
        accuracy_score(y_test, models["AdaBoost"].predict(X_test)),
        optimized_accuracy
    ]
})

display(final_results.round(4))

plt.figure(figsize=(8, 4))
plt.bar(final_results["Model"], final_results["Test Accuracy"])
plt.ylim(0, 1.05)
plt.ylabel("Test Accuracy")
plt.title("Model Performance Comparison")
plt.xticks(rotation=20, ha="right")
plt.tight_layout()
plt.show()

best_model_name = final_results.loc[final_results["Test Accuracy"].idxmax(),␣
  ↪"Model"]
print("Best model based on test accuracy:", best_model_name)
```

	Model	Test Accuracy
0	SVM (Before)	0.9667
1	Random Forest	0.9000
2	AdaBoost	0.9333
3	SVM (After Grid Search)	0.9333



```
Best model based on test accuracy: SVM (Before)
```

Conclusion

SVM, Random Forest, and AdaBoost were trained and compared on the Iris dataset. Five-fold cross-validation was used to estimate model performance, and Grid Search was used to optimize SVM hyperparameters. Test accuracy before and after optimization was compared. Optimization may improve accuracy, leave it unchanged, or reduce it slightly; the measured result should be reported rather than assuming improvement.